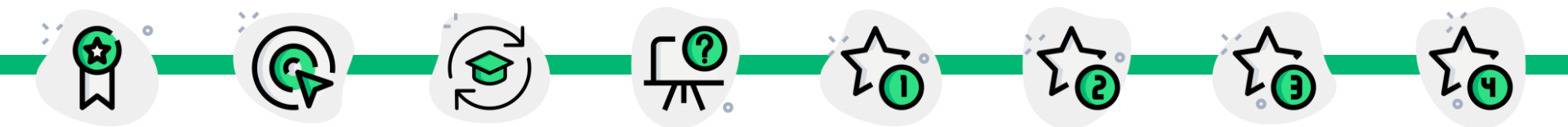


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.03.09

1주차



2026학년도 1학기 수업계획서

강좌명	알고리즘						담당교수	박 상 민	
학수번호	IC176						연구실	복음관 5층 512호	
학 점	3		이수구분		전공선택		연 락 처	02) 950 - 5597	
강의유형	이론	3	실습		설계		E-MAIL	smpark@bible.ac.kr	
수업시간	월요일 오후 16:20 ~ 17:35 (1교시) 17:45 ~ 19:00 (2교시)						면담시간	면담 전 사전 신청	

4. 성적평가						
방 법		출석	중간고사	기말고사	과제물	퀴즈
비율(%)		10	30	30	20	10
평가 세부 사항	과제물 평가 기준	○ 과제물 평가기준은 난이도, 독창성과 창의성의 정도에 따라서 다음과 같이 점수를 차등으로 반영함. (1) 중간 실습과제(10점), 기말 실습과제(10점)으로 구분 : 0점 ~ 10점(상:10~8, 중:7~5, 하:4~1)까지 반영하고, 제출하지 않은 경우 0점 처리하며, 제출기일을 초과해서 제출한 경우는 해당점수에서 -1점을 감점함) (2) 다른 학생의 결과물을 그대로 복사하여 제출한 경우, 관련된 학생들 모두 0점 처리함. (3) 보고서 내용: - 표지: 과제물 제목, 제출일시, 담당교수명, 학생명, 학번 - 목차: 각 장의 제목 및 페이지 번호 - 내용에는 서론, 본론, 결론, 배운점 - 참고문헌 및 부록				
	퀴즈	○ 퀴즈 (10%) - 총 15주차 수업 중 5번의 퀴즈 진행(퀴즈는 해당 주차 수업 시간 내에 제출해야 함) - 퀴즈 일정은 9. 주차별 수업계획 참고				
	출결 평가 기준	○ 3회 지각 및 조퇴는 결석 1회로 처리 ○ 결석 1회시 1점 감점 ○ 총 수업시간의 1/5 이상 결석한 자는 과목 성적 "F" 처리				
	기타	○ 법정 공휴일과 학교의 공식행사(Faith Training, 밀알축제)로 인해 수업일수가 15주 미만인 경우, 학생들과 시간을 조정하여 보강할 계획이며, 차후 공지할 것임				

9. 주차별 수업계획				
주	일자	단원명	강의내용	교수학습방법, 평가방법 및 과제물 (실습, 이론 등)
1주	3/9	교과목 오리엔테이션	- 교과목 오리엔테이션	- 이론 강의
			- 알고리즘이란? - 반복문	- 이론 강의
2주	3/16	배열, 포인터와 구조체	- 배열이란?	- 이론 강의
			- 포인터 - 구조체	- 이론 강의 - 퀴즈
3주	3/23	검색 알고리즘	- 검색 알고리즘이란 - 선형검색	- 이론 강의
			- 이진검색	- 이론 - 자기주도적 학습 과제물(본 학습 을 통하여 자기주도적으로 문제 해결을 하기 위한 능력을 키움)
4주	3/30	스택과 큐	- 스택이란? - 스택 구조체 - 스택 함수 알아보기	- 이론 강의
			- 큐란? - 큐 구조체 - 큐 함수 알아보기	- 이론 강의 - 퀴즈
5주	4/6	재귀 알고리즘(1/2)	- 재귀의 기본 - 순차곱셈, 유클리드 호제법	- 이론 강의
			- 재귀 알고리즘 분석하기	- 이론 강의
6주	4/13	재귀 알고리즘(2/2)	- 하노이탑의 원리 및 구현	- 이론 강의
			- 8퀸 문제의 원리 및 구현	- 이론 강의 - 자기주도적 학습 과제물(본 학습 을 통하여 자기주도적으로 문제

7주	4/20	정렬 알고리즘(1/3)	- 정렬이란? - 버블 정렬	- 이론 강의 및 실습 - 하이브리드러닝
			- 단순 선택 정렬 - 단순 삽입 정렬	- 이론 강의 및 실습 - 하이브리드러닝
8주	4/27	1교시: 정리 및 리뷰	- 중간고사 전 리뷰	- 정리 및 리뷰
		2교시: 중간고사	- 지필고사	- 중간고사
9주	5/4	정렬 알고리즘(2/3)	- 셸 정렬	- 이론 강의 및 실습
			- 퀵 정렬	- 이론 강의 및 실습 - 퀴즈
10주	5/11	정렬 알고리즘(3/3)	- 병합 정렬	- 이론 강의 및 실습
			- 힙 정렬 - 도수 정렬	- 이론 강의 및 실습 - 자기주도적 학습 과제물(본 학습을 통하여 자기주도적으로 문제 해결을 하기 위한 능력을 키움)
11주	5/18	문자열 검색	- 문자열의 기본 - 브루트 포스법	- 이론 강의 - 하이브리드러닝
			- KMP법 - Boyer-Moore법	- 이론 강의 - 하이브리드러닝
12주	5/25	오프라인 보강	오프라인 보강 6/15 월요일 진행 * 대체공휴일 (5/25)	- 보강
13주	6/1	리스트	- 선형 리스트 - 포인터로 연결 리스트 만들기	- 이론 강의
			- 커서로 연결 리스트 만들기 - 원형 이중 연결 리스트 만들기	- 이론 강의 - 퀴즈
14주	6/8	트리	- 트리란? - 트리관련 용어 - 순서트리의 순회 방법(전위/중위/후위순회)	- 이론
			- 이진트리와 이진검색트리	- 이론
15주	6/15	해시법 보강 주 보강	- 해시법 기본 개념 - 보강 진행(5/25 수업에 대한 보강)	- 이론
			- 해시법 충돌 및 대처 방법 - 보강 진행(5/25 수업에 대한 보강)	- 이론 - 실제적 과제(본 학습을 통하여 실제 현장 문제 해결에 어떠한 효과가 있는지 분석함)
16주	6/22	1교시: 정리 및 리뷰	- 기말고사 전 리뷰 (75분)	- 정리 및 리뷰
		2교시: 기말고사	- 지필고사 (75분)	- 기말고사

◆ 강의 자료

- 강의자료는 강의가 진행되는 주에 LMS에서 삭제 예정 → 꼭 해당 주에 다운로드 받을 것!
- 업로드 된 강의자료는 배포용 강의자료임. 그림과 표 등의 자료는 첨부되어 있지 않을 수 있음.

알고리즘이란?



01-1 알고리즘이란?

◆ 간단한 예시로 알고리즘에 대하여 알아보자

- 아래 예시는 세 정수의 최대값을 구하는 프로그램임.

Chapt01_예제 1-1/max3.c

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main(void)
{
    int a, b, c;

    printf("세 정수의 최대값을 구합니다.\n"); //문자 출력
    printf("a값: "); scanf("%d", &a); // a값 입력
    printf("b값: "); scanf("%d", &b); // b값 입력
    printf("c값: "); scanf("%d", &c); // c값 입력

    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;

    printf("최대값은 %d입니다.\n", max);

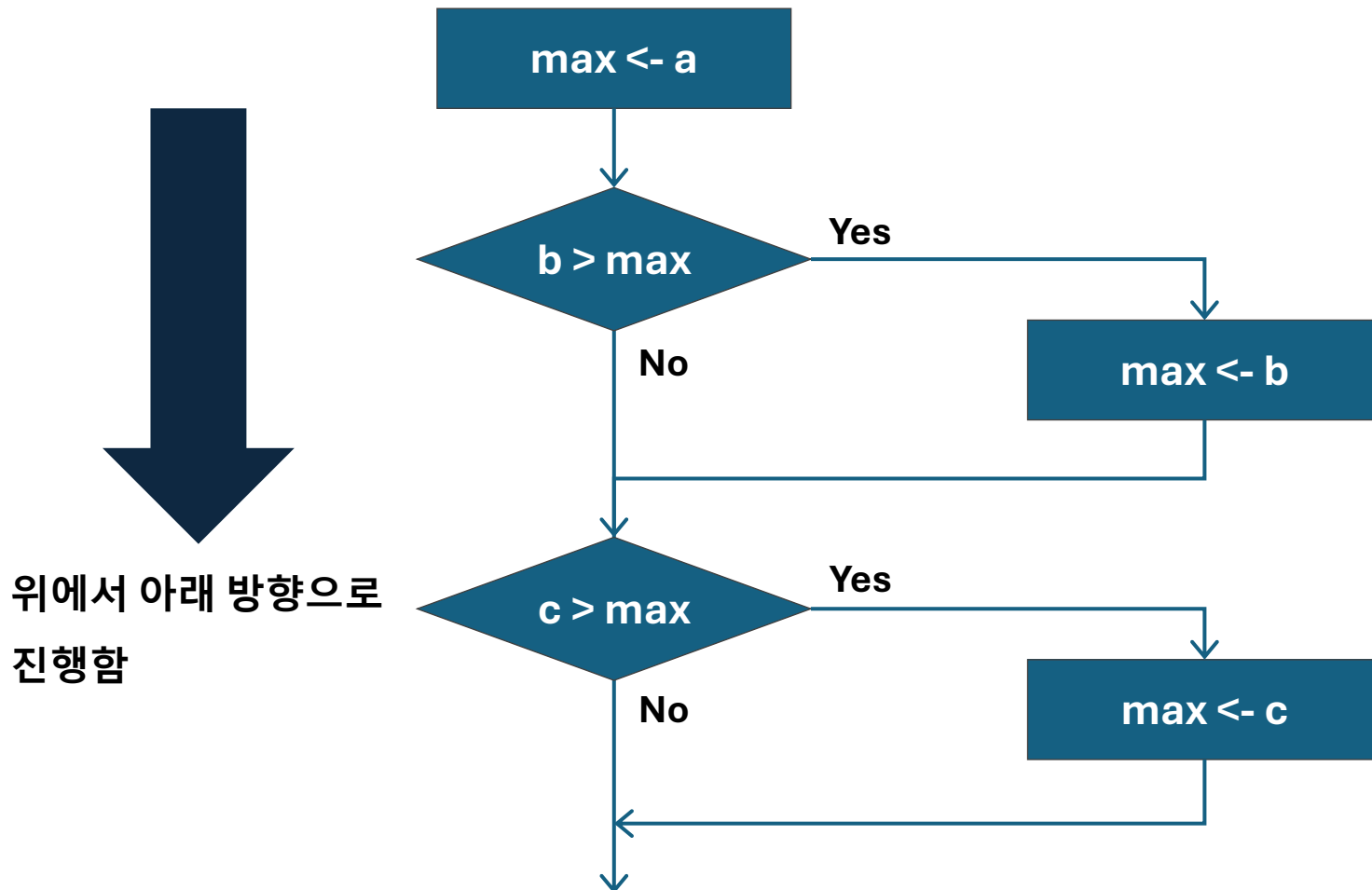
    return 0;
}
```




01-1 알고리즘이란?

◆ 세 정수의 최대값을 구하는 알고리즘의 순서도

- 아래 예시는 세 정수의 최대값을 구하는 알고리즘의 순서도(flow chart).





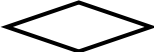
01-1 알고리즘이란?

◆ 세 정수의 최대값을 구하는 알고리즘의 순서도

■ 세 정수의 최대값을 구하는 알고리즘 순서도

✓ 알고리즘의 흐름은 위에서 아래 방향

✓  : 처리 과정 실행

✓  : 조건을 평가한 결과에 따라
'Yes'나 'No' 중 하나를 선택

✓ 알고리즘(또는 프로그램)이 흐르는 방향은 '조건식'이 결정함.

✓ ' $\text{max} \leftarrow a$ ' 의미: 변수 a값을 변수 max에 대입하세요.

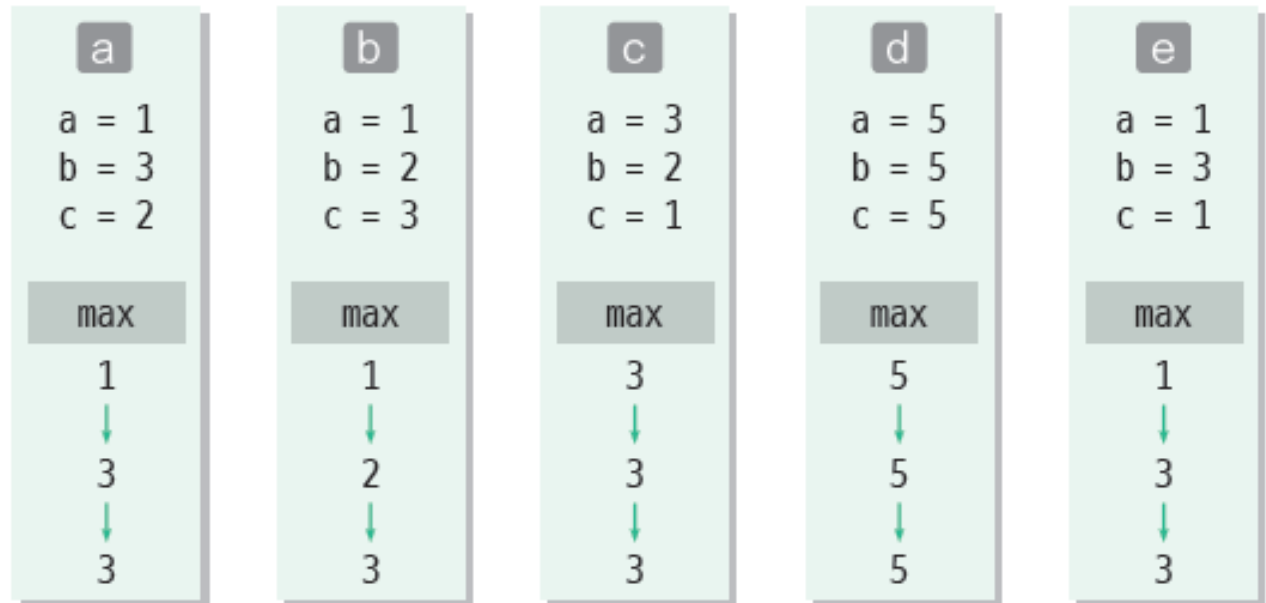


01-1 알고리즘이란?

◆ 세 정수의 최대값을 구하는 알고리즘의 순서도

- 세 정수의 최대값을 구하는 과정에서 max값의 변화

```
max = a;  
if(b > max) max = b;  
if(c > max) max = c;
```



[그림 1-2] 세 정수의 최대값을 구하는 과정에서 변수 max값의 변화



01-1 알고리즘이란?

◆ 알고리즘의 정의

■ 알고리즘의 정의

어떤 문제를 해결하기 위한 절차로, 명확하게 정의되고 순서가 있는
유한 개의 규칙으로 이루어진 집합

■ 올바른 알고리즘이란?

- ✓“어떠한 경우에도 실행 결과가 똑같이 나오는 것”을 말한다.
- ✓만약 알고리즘의 실행 결과가 어떤 경우에는 맞고 어떤 경우에는 틀리면 올바른 알고리즘이라고 할 수 없다.



01-1 알고리즘이란?

◆ 조건 판단과 분기 살펴보기

Chapt01_예제 1-3/sign.c

// 입력받은 정숫값의 부호(양수/음수/0)를 판단

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
```

```
int main(void)
{
    int n;

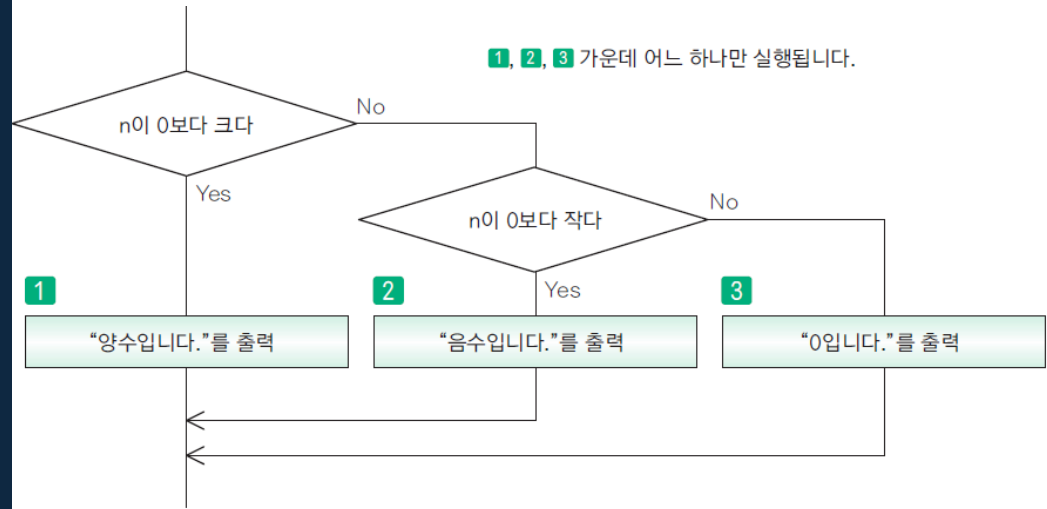
    printf("정수: ");
    scanf("%d", &n);

    if (n > 0)
        printf("양수입니다.\n");
    else if (n < 0)
        printf("음수입니다.\n");
    else
        printf("0입니다.\n");

    return 0;
}
```

실습 1-3

- ✓ 입력한 정숫값의 부호(양수/음수/0)를 판단하여 출력하는 프로그램
- ✓ n값이 양수면 **1**, 음수면 **2**, 0이면 **3** 이 실행 프로그램의 흐름이 세 가지로 분기





01-1 알고리즘이란?

◆ 조건 판단과 분기 살펴보기 (2)

■ 예제 1-4

- $n = 1$ 또는 2가 아니면 C 라고 출력함
- 프로그램 ➔ 세 가지로 분기됨

(... 생략 ...)

```
12  if(n == 1)
13      puts("A");
14  else if(n == 2)
15      puts("B");
16  else
17      puts("C");
```

(... 생략 ...)

• 완성 파일 chap01/judge1.c

실행 결과

1 정수: 3
C
2 정수: 4
C

■ 예제 1-5

- n 이 1, 2, 3 이외의 수일 때는 아무것도 표시하지 않음
- 프로그램 ➔ 네 가지로 분기됨

(... 생략 ...)

```
12  if(n == 1)
13      puts("A");
14  else if(n == 2)
15      puts("B");
16  else if(n == 3)
17      puts("C");
```

(... 생략 ...)

• 완성 파일 chap01/judge2.c

실행 결과

1 정수: 3
C
2 정수: 4



01-1 알고리즘이란?

◆ 조건 판단과 분기 살펴보기 (3)

■ 예제 1-6

- 실습 1-5에 숨겨진 '아무 것도 실행하지 않는' else 를 명시

(... 생략 ...)

```
12  if(n == 1)
13      puts("A");
14  else if(n == 2)
15      puts("B");
16  else if(n == 3)
17      puts("C");
18  else
19      ;
```

(... 생략 ...)

• 완성 파일 chap01/judge2x.c

실행 결과

정수: 3

C

정수: 4



01-1 알고리즘이란?

◆ 순서도 flowchart

■ 순서도 기호 살펴보기

- ✓문제에 대한 정의·분석·해법을 그림으로 표현
- ✓프로그램 순서도(program flowchart)에서 사용되는 기호

- 실제로 수행하는 연산을 나타내는 기호
- 제어의 흐름을 나타내는 기호
- 프로그램 순서도를 이해하고 작성하는 데 편의를 부여하는 특수 기호

용어	의미	기호
데이터 (data)	• 데이터의 입력과 출력	
처리 (process)	• 여러 종류의 처리 기능을 수행 • 정보의 값, 자료형, 위치를 바꾸도록 정의한 연산이나 연산군의 실행 • 연속적인 몇 가지 흐름 가운데 하나의 방향을 결정하는 연산이나 연산군의 실행	



01-1 알고리즘이란?

◆ 순서도 flowchart

▪ 순서도 기호 살펴보기

용어	의미	기호
미리 정의한 처리 (predefined process)	서브 루틴 및 모듈 등 다른 곳에서 이미 정의한 하나 이상의 연산 또는 명령어들로 이루어진 처리	
판단 (decision)	- 하나의 입구와 하나 이상을 선택할 수 있는 출구가 있고, 기호에서 정의한 조건을 평가하여 하나의 출구를 선택하는 판단 기능(스위치형 기능)을 나타냄 - 주로 예상되는 평가 결과의 경로를 선 가까이에 표시	
선 (line)	- 제어의 흐름 - 흐름의 방향을 분명히 나타내고자 할 때 화살표를 붙임	
단말 (terminator)	- 외부 환경으로 나가거나 외부 환경에서 들어오는 것을 나타냄 - 주로 프로그램 흐름의 시작과 종료를 나타냄	



01-1 알고리즘이란?

◆ 순서도 flowchart

▪ 순서도 기호 살펴보기

용어	의미	기호
루프 범위 (loop limit)	• 두 부분으로 구성되어 루프의 시작과 종료를 나타냄 • 기호의 두 부분에는 같은 이름(루프에 대한 임의의 이름)을 사용 • 루프의 시작 기호(반복 전에 판단하는 경우) 또는 종료 기호(반복 후에 판단하는 경우) 안에 초기값(초기화), 증가값, 종료값(종료 조건)을 표기	

구성	내용	모양
데이터	데이터 입출력	
처리	데이터를 처리한다.(연산이나 저장 등)	
판단	조건에 따라 참과 거짓에 결과에 따라 데이터의 흐름을 결정	
문서입력	데이터를 출력	
시작/끝	블록 다이어그램의 시작과 끝	
<u>흐름선</u>	처리의 흐름을 표시	
<u>연결자</u>	흐름의 연결을 표시	
미리 정의된 처리	미리 정의된 처리로 옮길 때 사용	
수작업 입력	키보드를 통한 입력	
준비	초기값 설정 등의 준비 단계를 표시	

반복문



01-2 반복

◆ 순서도 flowchart

Chapt01_예제 1-7/sum_while.c

// 1~n의 합을 구하여 출력(while문)

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n;
```

```
    puts("1부터 n까지의 총합을 구합니다.");
```

```
    printf("n값: ");
```

```
    scanf("%d", &n);
```

```
    int sum = 0;    // 총합
```

```
    int i = 1;
```

```
    while (i <= n) {    // i가 n 이하이면 반복
```

```
        sum += i;    // sum에 i를 추가
```

```
        i++;    // i값을 1 증가
```

```
    }
```

```
    printf("1부터 %d까지의 총합은 %d입니다.\n", n, sum);
```

```
    return 0;
```

```
}
```

1부터 n까지 정수의 합 구하기 - (1)

■ 예제 1-7

- 1부터 n까지의 정수 합을 구하는 알고리즘

■ 반복 구조 repetition structure 또는 루프 loop

- 어떤 조건이 성립하는 동안 처리(프로그램 명령문 또는 명령어의 집합)를 반복해 실행하는 것

■ while(제어식) 명령문

- 반복의 대상이 되는 '명령문'을 문법적으로는 '루프 본문'이라 함

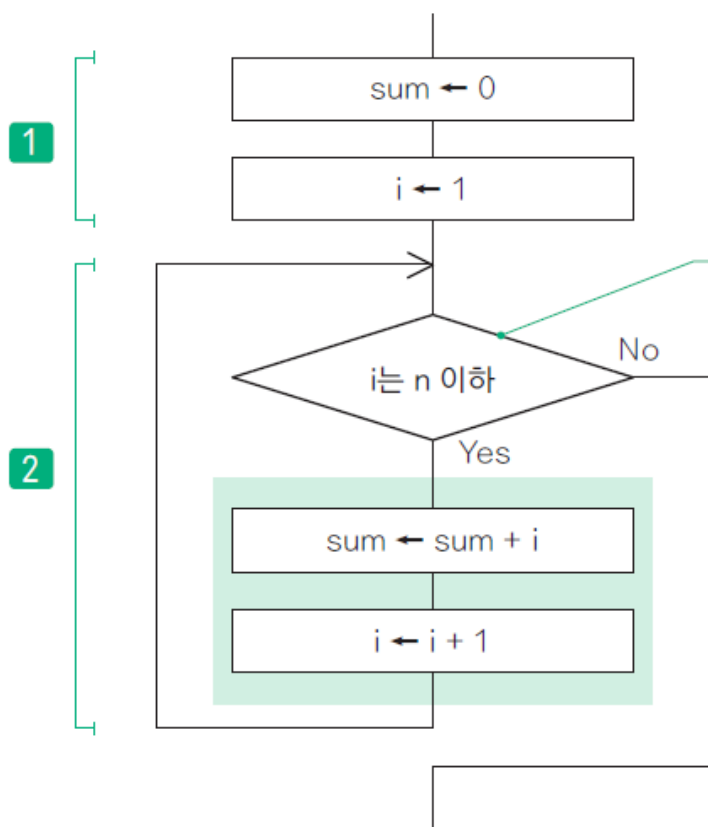
■ 사전 판단 반복

- 실행 전에 반복을 계속할지를 판단
- 제어식의 평가값이 0이 아니면 프로그램 명령문이 반복



01-2 반복

◆ 순서도 flowchart



```
int sum = 0; // 총합
```

```
int i = 1;
```

```
while (i <= n) { // i가 n 이하이면 반복
```

```
    sum += i; // sum에 i를 추가
```

```
    i++; // i값을 1 증가
```

여기를 지나면 i와 sum의 값이 변화합니다.

i	sum
1	0
2	1
3	3
4	6
5	10
6	15
⋮	⋮

1을 더합니다.

2을 더합니다.

3을 더합니다.

4을 더합니다.

5을 더합니다.

1~5의 합

[그림 1-12] 1부터 n까지의 합을 구하는 순서도와 변수의 변화



01-2 반복

◆ 순서도 flowchart

Chapt01_예제 1-8/sum_for1.c

// 1, 2, ..., n의 값을 구하여 출력(for 문)

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n;
```

```
    puts("1부터 n까지의 총합을 구합니다.");
```

```
    printf("n값: ");
```

```
    scanf("%d", &n);
```

```
    int sum = 0;
```

```
    for (int i = 1; i <= n; i++) // i = 1, 2, ..., n
        sum += i;             // sum에 i를 추가
```

```
    printf("1부터 %d까지의 총합은 %d입니다.\n", n, sum);
```

```
    return 0;
```

```
}
```

1부터 n까지 정수의 합 구하기 - (2)

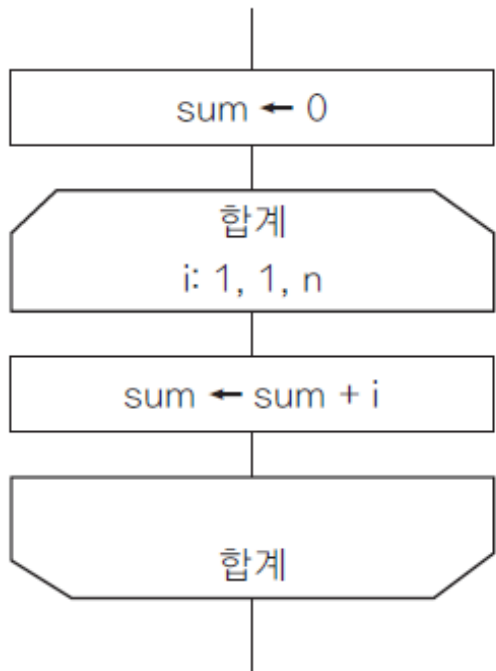
■ 예제 1-8

- 정수의 총합을 for 문으로 구하는 프로그램
- for(식1; 식2; 식3) 명령문 또는 for(선언; 식2; 식3) 명령문
 - 하나의 변수를 사용하는 반복문은 while 문보다 for 문을 사용하는 것이 좋음
 - 프로그램의 흐름이 for 문으로 바뀌면 식1이 평가 및 실행
 - 제어식인 식2를 평가한 값이 0이 아니면 루프 본문이 반복하여 실행됨
 - 그런 다음 식3이 평가, 실행



01-2 반복

◆ 순서도 flowchart



```
int sum = 0;
```

```
for (int i = 1; i <= n; i++) // i = 1, 2, ..., n  
    sum += i;               // sum에 i를 추가
```

i값을 1부터 시작하여 n이 될 때까지 1씩 증가하면서 반복합니다.

[그림 1-13] 1부터 n까지의 합을 구하는 순서도



01-2 반복

◆ 순서도 flowchart

Chapt01_예제 1-9/sum_for2.c

// 1, 2, ..., n의 합을 구하여 출력(do 문에서 양수만 n값으로 입력)

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n;
```

```
    puts("1부터 n까지의 총합을 구합니다.");
```

```
    do {
```

```
        printf("n값: ");
```

```
        scanf("%d", &n);
```

```
    } while (n <= 0);
```

```
    int sum = 0;    // 총합
```

```
    for (int i = 1; i <= n; i++) // i = 1, 2, ..., n
```

```
        sum += i;    // sum에 i를 추가
```

```
    printf("1부터 %d까지의 총합은 %d입니다.\n", n, sum);
```

```
    return 0;
```

```
}
```

양수만 입력하기-(2)

■ 예제 1-9

- 양수만 입력 받아 1부터 n까지 정수의 합을 구하는 프로그램

■ do 명령문 while(제어식)

■ 사후 판단 반복

- 일단 루프 본문을 한 번 실행한 다음에 계속 반복할 것인지를 판단
- () 안의 제어식을 평가한 값이 0이 아니면 루프 본문의 명령문이 반복

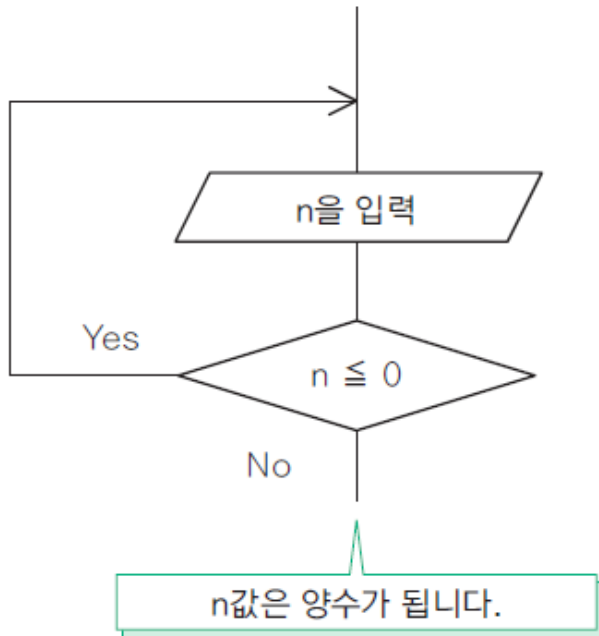


01-2 반복

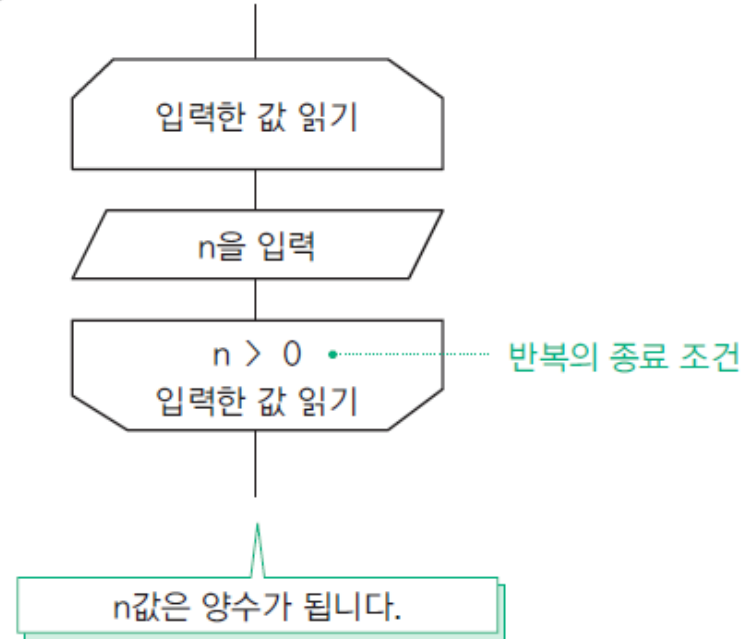
◆ 순서도 flowchart

```
do{  
    printf("n값: ");  
    scanf("%d", &n);  
}while (n <= 0);
```

a



b



[그림 1-14] 양수를 입력하고 입력한 값을 읽는 순서도



01-2 반복

◆ 순서도 flowchart

Chapt01_예제 1-10/multi99table.c

// 곱셈표를 출력

```
#include <stdio.h>

int main(void)
{
    printf("----- 곱셈표 ----- \n");

    for (int i = 1; i <= 9; i++) {
        for (int j = 1; j <= 9; j++)
            printf("%3d", i * j);
        putchar('\n');
    }

    return 0;
}
```

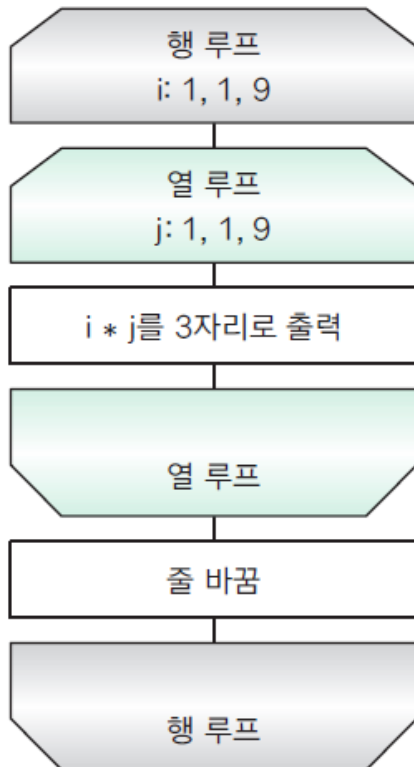
다중 루프 다루기 - (1)

- 다중 루프
 - 반복 안에서 다시 반복할 수 있음
 - 이런 반복을 루프가 중첩되는 수준에 따라 '이중 루프, 삼중 루프'라고 함
- 실습 1-10
 - 이중 루프로 구구단 곱셈표를 출력하는 프로그램
 - 행 루프 : 바깥쪽의 for 문은 세로 방향에 대한 반복으로, 표의 '1행, 2행, ..., 9행'에 해당
 - 열 루프 : 안쪽의 for 문은 가로 방향에 대한 반복으로, 표의 '1열, 2열, ..., 9열'에 해당



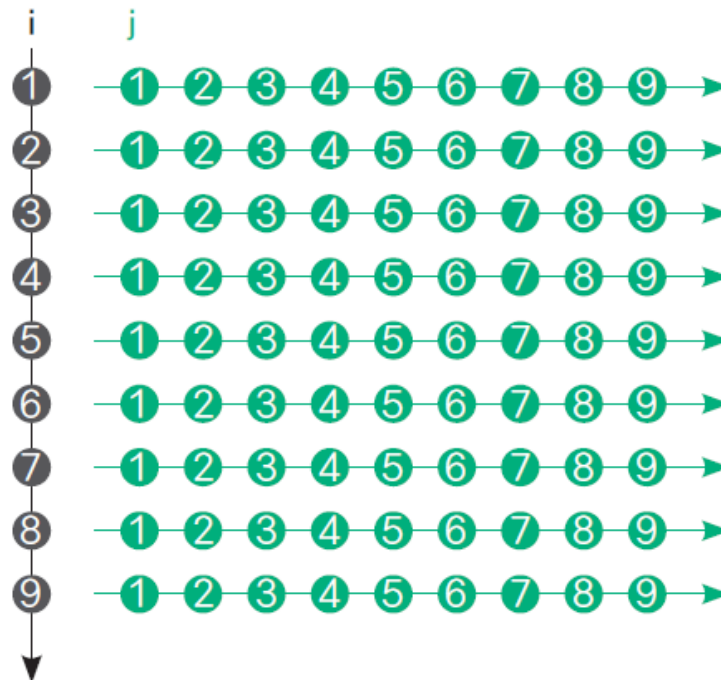
01-2 반복

◆ 순서도 flowchart



```
for (int i = 1; i <= 9; i++) {  
    for (int j = 1; j <= 9; j++)  
        printf("%3d", i * j);  
    putchar('\n');  
}
```

변수 i와 j의 변화



[그림 1-15] 곱셈표를 출력하는 순서도



01-2 반복

◆ 순서도 flowchart

Chapt01_예제 1-11/triangleLB.c

// 왼쪽 아래가 직각인 이등변 삼각형을 출력

```
#include <stdio.h>
```

```
int main(void)
{
    int n;
    do {
        printf("몇 단 삼각형입니까?: ");
        scanf("%d", &n);
    } while (n <= 0);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            putchar('*');
        putchar('\n');
    }

    return 0;
}
```

다중 루프 다루기 - (2)

■ 예제 1-11

- 왼쪽 아래가 직각인 이등변 삼각형을 출력하는 프로그램
- 07~10행은 변수 n에 입력하는 값을 양수로 제한하려는 do 문
- 행 루프 : 바깥쪽 for 문은 삼각형의 각 행에 대응하는 세로 방향 반복
- 열 루프 : 안쪽 for 문은 삼각형의 각 열에 대응하는 가로 방향의 반복

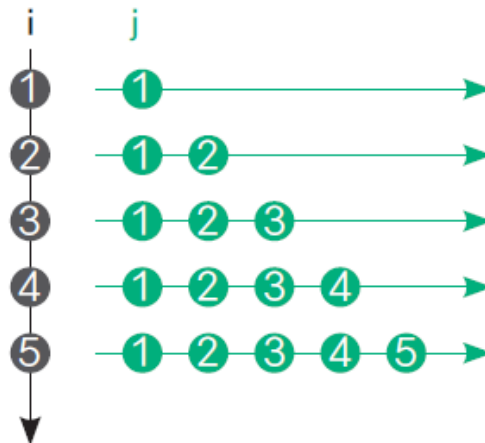


01-2 반복

◆ 순서도 flowchart



변수 i와 j의 변화(n은 5로 합니다).



```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++)  
        putchar('*');  
    putchar('\n');  
}
```

[그림 1-16] 왼쪽 아래가 직각인 이등변 삼각형을 출력하는 순서도

「2026년도 1학기 한국성서대학교 AI융합학부」

감사합니다.

2026.03.09

1주차

